

Lab 1: Arduino Foundations

Blink, External LED, and Push-Button Input

ENGR 1105 | Fall 2026

Name	Partner
Date	Section

Learning Objectives

By the end of this lab, you should be able to:

1. connect an Arduino Uno and upload a program;
2. modify the built-in Blink example;
3. build a safe external LED circuit;
4. read a push button using `INPUT_PULLUP`;
5. use an `if` statement to control an output; and
6. troubleshoot simple wiring and code problems.

1. Lab Organization

Lab 1 is completed in two class meetings:

Part	Main work
Lab 1.1	Arduino connection, built-in Blink, modified timing, and external LED
Lab 1.2	Push-button input, decision logic, troubleshooting, and design challenge

2. Materials

Item	Purpose
Arduino Uno	Controller
USB data cable	Power and program upload
Solderless breadboard	Temporary circuit construction
Jumper wires	Electrical connections
LED	Visual output
330 Ω resistor	Limits LED current
Push button	User input
Computer with Arduino IDE	Code editing and upload

Before Applying Power

1. Disconnect USB power before making major wiring changes.
2. Never connect 5V directly to GND.
3. Use a resistor in series with every external LED.
4. Check LED polarity before reconnecting power.
5. Remove power immediately if the Arduino becomes hot or repeatedly disconnects.

Lab 1.1

Arduino Setup and Digital Output

Built-In Blink and External LED

3. Part A: Connect the Arduino

Instructions

1. Connect the Arduino Uno to the computer using the USB data cable.
2. Open the Arduino IDE.
3. Select **Tools > Board > Arduino Uno**.
4. Select the correct port under **Tools > Port**.
5. Open **File > Examples > 01.Basics > Blink**.
6. Click **Verify**.
7. Click **Upload**.

Checkpoint 1: Board Connection

Show the instructor that:

- the correct board and port are selected;
- the program uploads without an error; and
- the built-in LED blinks.

Observation 1

How long does the built-in LED stay ON? How long does it stay OFF?

Approximately how long is one complete ON-OFF cycle?

4. Part B: Modify Blink

The value inside `delay()` is measured in milliseconds.

Instructions

1. Change both delay values to 1000.
2. Upload the program and observe the LED.
3. Change both delay values to 200.
4. Upload the program and observe the LED again.
5. Create one additional blink pattern of your choice.

Timing Results

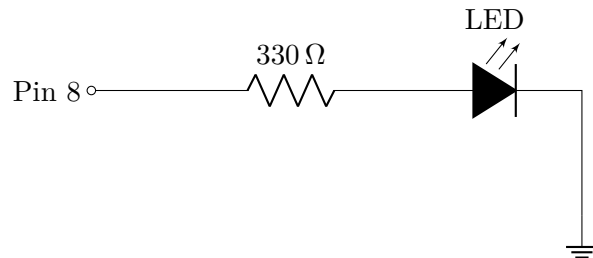
ON delay	OFF delay	Observed behavior
1000 ms	1000 ms	
200 ms	200 ms	
My pattern:		

Checkpoint 2: Modified Blink

Demonstrate your custom blink pattern and explain how the delay values control the timing.

5. Part C: Build an External LED Circuit

Use Arduino digital pin 8.

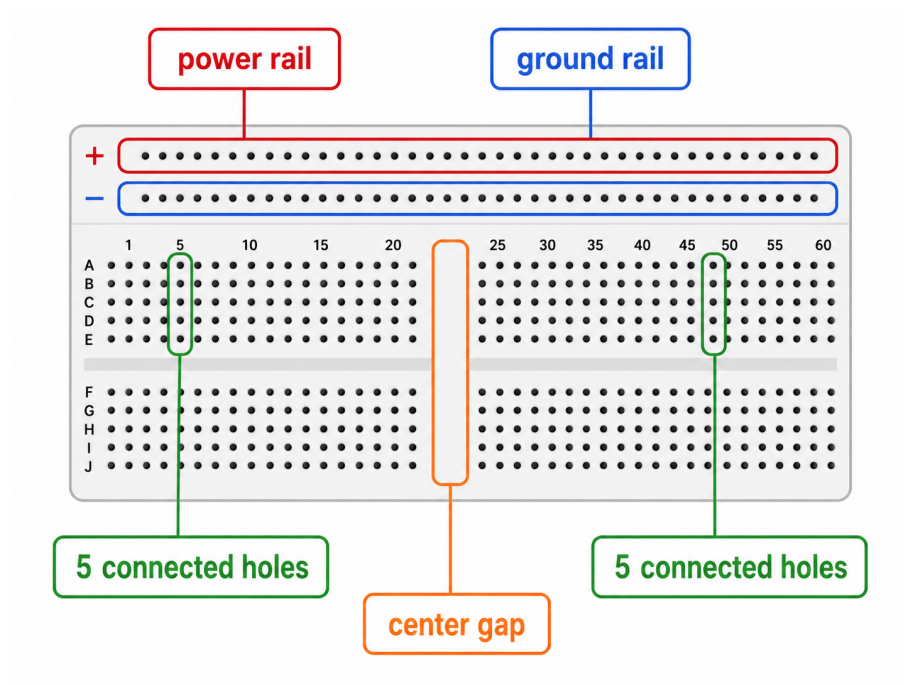


Wiring Steps ---

1. Disconnect the USB cable.
2. Place the LED on the breadboard so the two leads are in different terminal groups.
3. Connect Arduino pin 8 to one end of the $330\ \Omega$ resistor.
4. Connect the other end of the resistor to the LED's long lead.
5. Connect the LED's short lead to Arduino GND.
6. Ask for a wiring check before reconnecting USB power.

LED Polarity ---

The longer LED lead is normally the anode. The shorter lead and flat edge mark the cathode. The resistor must be in series with the LED.



6. Part D: Program the External LED

```
1  const int ledPin = 8;
2
3  void setup() {
4      pinMode(ledPin, OUTPUT);
5  }
6
7  void loop() {
8      digitalWrite(ledPin, HIGH);
9      delay(500);
10
11     digitalWrite(ledPin, LOW);
12     delay(500);
13 }
```

Instructions

1. Type or copy the program into a new Arduino sketch.
2. Verify the program.
3. Upload it to the Arduino.
4. Confirm that the external LED blinks.
5. Change the timing and upload the program again.

Checkpoint 3: External LED

Demonstrate the external LED blinking. Point to:

- Arduino pin 8;
- the $330\ \Omega$ resistor;
- the LED anode and cathode; and
- the ground connection.

Concept Check

Why is the resistor needed?

What would happen if the LED were installed backward?

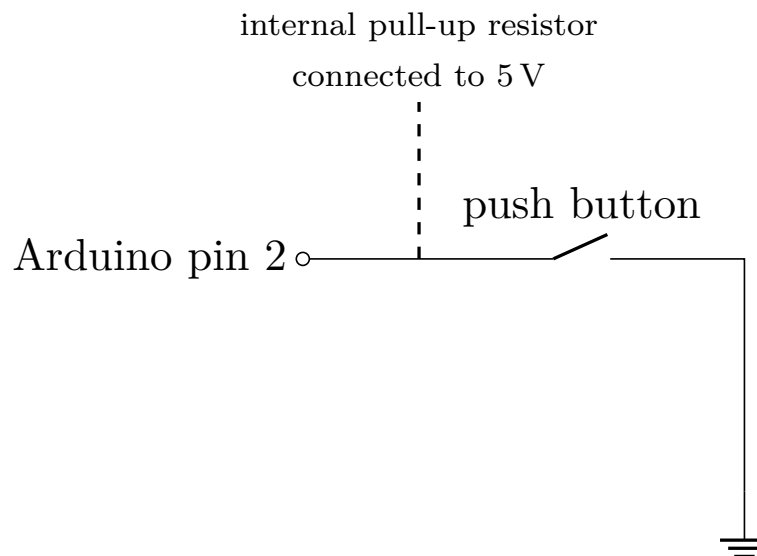
Lab 1.2

Push-Button Input and Decisions

Using INPUT_PULLUP

7. Part E: Wire the Push Button

Use Arduino digital pin 2.



Wiring Steps

1. Disconnect USB power.
2. Place the push button across the breadboard center gap.
3. Connect one side of the button to Arduino pin 2.
4. Connect the opposite side of the button to Arduino GND.
5. Keep the external LED connected to pin 8.
6. Ask for a wiring check before reconnecting power.

Button Orientation

A four-leg push button has two internally connected pairs of legs. Incorrect orientation can make the input appear permanently pressed or permanently released.

8. Part F: Read the Button

```
1  const int buttonPin = 2;
2  const int ledPin = 8;
3
4  void setup() {
5      pinMode(buttonPin, INPUT_PULLUP);
6      pinMode(ledPin, OUTPUT);
7  }
8
9  void loop() {
10     int buttonState = digitalRead(buttonPin);
11
12     if (buttonState == LOW) {
13         digitalWrite(ledPin, HIGH);
14     } else {
15         digitalWrite(ledPin, LOW);
16     }
17 }
```

Test the Program

1. Upload the program.
2. Observe the LED when the button is not pressed.
3. Press and hold the button.
4. Observe the LED while the button is pressed.
5. Release the button.
6. Confirm that the LED returns to its original state.

Observation 2

Complete the table.

Button state	Input reading	LED state
Not pressed		
Pressed		

Why does the input become LOW when the button is pressed?

Checkpoint 4: Button-Controlled LED

Demonstrate that the button controls the LED. Explain:

- what `digitalRead()` does;
- what the `if` statement checks; and
- why pressed means LOW.

9. Troubleshooting

Problem	Check first
Program will not upload	Correct board, correct port, and USB data cable
Built-in LED does not blink	Upload status and correct board selection
External LED does not light	LED polarity, resistor placement, pin 8, and ground
Button always reads one value	Button orientation, pin 2, ground, and INPUT_PULLUP
Arduino becomes hot	Disconnect immediately and inspect for a short circuit

10. Part G: Design Challenge

Design Challenge

Choose one challenge:

1. Make the LED blink rapidly only while the button is pressed.
2. Make the LED normally ON and turn OFF while the button is pressed.
3. Use a slow blink when the button is not pressed and a fast blink when it is pressed.

Design Record

Which challenge did you choose?

Describe the code changes you made.

What happened when you tested the program?

11. Final Reflection

Explain how this lab demonstrates the sense-decide-act model.

12. Submission Checklist

- ☐ Completed Lab 1 instruction sheet
- ☐ Clear photo of the final circuit
- ☐ Final Arduino code
- ☐ Completed design challenge and reflection